

Leftover properties of **this** in constructor preconditions

Document #: P3510R1
Date: 2024-11-20
Project: Programming Language C++
Audience: SG21
Reply-to: Nathan Myers
<ncm@cantrip.org>
Gašper Ažman
<gasper.azman@gmail.com>

Contents

- 1 Abstract
- 2 Motivation
- 3 Background
- 4 Analysis
 - 4.1 Difference with initialization lists
- 5 Proposal
- 6 Wording
- 7 References

§ 1 Abstract

In preconditions on constructors and postconditions on destructors, all the non-static data members of the class are uninitialized, and accessing them is undefined behaviour.

We therefore propose disallowing implicit access to non-static class members to reduce the risk of such use, as a change to [\[P2900R10\]](#).

§ 2 Motivation

We have a policy of not gratuitously introducing new kinds of UB in contract contexts, as we seek to improve reliability and various kinds of safety, not undermine them.

The following code is well-formed under [\[P2900R10\]](#), but evokes UB:

```

class X
{
    std::string name;

public:
    explicit X(const char * n)
        pre(name != nullptr) // evaluated before
        : name{n}             // evaluated during
    {}
};

```

This code might be recognized from [P3172R0], from 2024-03-08, adopted at the lightly-attended Tokyo meeting. This use of the uninitialized member name in the uninitialized `*this` is language UB. While we may expect compilers to issue a stern warning for such misuse, warnings are often not seen, and as often are not understood. No one has suggested a reason to allow implicit mention of non-static, uninitialized members in this context.

§ 3 Background

Permitting access to the about-to-be constructed object's storage in contract assertions on constructors is motivated by a desire to operate on addresses of subobjects, particularly mixins, that may need to be checked before initializing the object. Typically, the address of interest is derived by casting; `static_cast<Mixin>(this)` provides a pointer value that may be offset from `this` by a compile-time constant value. Giving `this` in this context its more-natural `const void*` type would make this usage impossible in `constexpr` contexts: producing the needed X pointer would require a `reinterpret_cast<const X*>(this)`, an operation forbidden to `constexpr` code.

Similar access to addresses of members by implicit naming is not needed, and mention of such members is a source of unnecessary UB.

[P3172R0] goes on to provide access to such pointers in contract assertions on destructors, for more-or-less analagous reasons, and with a similar (albeit perhaps less attractive) hazard.

§ 4 Analysis

How did we get here? [P3172R0] notes the UB hazard, in its section 2.3, and asserts that containing the problem is unimplementable. It uses the example of passing `this` to a function that may then improperly dereference the pointer, undetectably to the compiler.

We do not find this example compelling; the UB in the called function is the responsibility of that function, but our responsibility is to prevent a very obvious source of accidental UB in precondition assertions themselves.

Mentioning `this` is very rare in typical users' code, and attracts attention, where mentioning members by implicit reference, UB here, is extremely common, and does not.

We note further that mentioning non-static member functions in this context would also invite UB, as the implicit `this` passed to such member functions is almost always used.

§ 4.1 Difference with initialization lists

Initialization lists do allow implicit access to non-static class members. It is up to the programmer to use only those data members that have so far been constructed. This unfixable source of potential UB is unfortunate, but there is anyway limited temptation to mention other members in that context.

Contrast this to constructor preconditions: no data members have been constructed at that point, so none may safely be used. Requiring an explicit `this->`, if they must be mentioned, is not too onerous of a work-around in the view of the authors, should the programmer really need an uninitialized member's address.

§ 5 Proposal

The solution is simple: make implicit access to non-static class members (both data and function) ill-formed in constructor preconditions and destructor postconditions, while preserving access to `this` itself.

§ 6 Wording

Add to [dcl.contract.func]

A predicate of a precondition assertion of a constructor, and a predicate of a postcondition assertion on a destructor, shall not make implicit reference to a non-static member or sub-object (i.e. without `this->`).

§ 7 References

[P2900R10] Joshua Berne, Timur Doumler, Andrzej Krzemiński. 2024-10-13. Contracts for C++.

<https://wg21.link/p2900r10>

[P3172R0] Andrzej Krzemiński. 2024-03-08. Using `this` in constructor preconditions.

<https://wg21.link/p3172r0>